

UNESCO Open Science Portal

Description	Documentation for the UNESCO Open Science Portal powered by InvenioRDM
Author(s)	UNESCO
Copyright	© Copyright 2021-2026, UNESCO.

Table of Contents

- 1 Getting started
- 2 Complete Guide
- 3 Kubernetes Deployment
- 4 Production Deployment

1 UNESCO Open Science Portal

Welcome to the UNESCO Open Science Portal powered by InvenioRDM - a modern research data management platform.

1.1 Overview

This repository contains:

- **InvenioRDM instance:** Full-featured research data repository
- **OpenScience Tools:** Python SDK and CLI for programmatic access to InvenioRDM
- Insert, update, delete records via SDK
- Batch import from Lens.org
- Search and manage records

1.2 Prerequisites

Before setting up the UNESCO Open Science Portal, ensure your system meets these requirements:

1.2.1 System Requirements

- **Operating System:** macOS, Linux, or Windows with WSL2
- **Memory:** Minimum 4GB RAM (8GB recommended for optimal performance)
- **Storage:** At least 5GB free disk space
- **Internet:** Stable connection for downloading dependencies

1.2.2 Required Software

- **Python 3.9+:** Required for InvenioRDM

```
python3 --version # Should show Python 3.9 or higher.
```

- **Docker & Docker Compose:** For containerized services

```
docker --version && docker compose version
```

- **Git:** For version control

```
git --version
```

1.2.3 Port Availability

Ensure these ports are available (not used by other applications):

- **5000**: Development server
- **5432**: PostgreSQL database
- **9200**: OpenSearch
- **6379**: Redis cache
- **5672**: RabbitMQ message queue

1.3 Quick Start

1.3.1 1. Clone the Repository

```
git clone <repository-url>  
cd sc-openscience
```

1.3.2 2. Start the Development Environment

```
# Start all services  
docker-compose up -d  
  
# Check service status  
docker-compose ps
```

1.3.3 3. Access the Portal

Once services are running, access:

- **Portal**: <http://localhost:5000>
- **Admin Panel**: <http://localhost:5000/administration>

1.4 OpenScience Tools

The OpenScience Tools provide a Python SDK and CLI for programmatic access to InvenioRDM.

1.4.1 Installation

```
cd openscience_tools  
pip install -e .
```

1.4.2 Usage Examples

SDK Usage

```
from openscience_tools import OpenScienceClient

# Initialize client
client = OpenScienceClient(base_url="http://localhost:5000", token="your-token")

# Create a record
record = client.create_record({
    "metadata": {
        "title": "My Research Data",
        "description": "Description of the research data"
    }
})

# Search records
results = client.search_records(query="research")
```

CLI Usage

```
# Search for records
openscience-cli search "research data"

# Import from Lens.org
openscience-cli import-lens --query "open science"

# Create a record
openscience-cli create-record --file record.json
```

1.5 Development

1.5.1 Project Structure

```
sc-openscience/
├─ app_data/          # Initial data and vocabularies
├─ assets/            # Frontend assets
├─ docker/           # Docker configurations
├─ openscience_tools/ # Python SDK and CLI
├─ site/              # Site-specific customizations
├─ static/            # Static files
└─ templates/         # Custom templates
```

1.5.2 Configuration

The main configuration file is `invenio.cfg`. You can customize:

- Site name and branding
- Authentication providers
- Storage backends

- Email settings
- Custom vocabularies

1.5.3 Running Tests

```
# Run OpenScience Tools tests
cd openscience_tools
pytest
```

1.6 Deployment

For production deployment instructions, see [DEPLOYMENT.md](#).

1.7 Support

For issues and questions:

- Create an issue in the repository
- Contact the development team

1.8 License

This project is licensed under the terms specified in the LICENSE file.

2 UNESCO Open Science Portal - Complete Guide

Welcome to the UNESCO Open Science Portal powered by InvenioRDM - a modern research data management platform.

2.1 Overview

This repository contains:

- **InvenioRDM instance:** Full-featured research data repository
- **OpenScience Tools:** Python SDK and CLI for programmatic access to InvenioRDM
- Insert, update, delete records via SDK
- Batch import from Lens.org
- Search and manage records

2.2 Prerequisites

Before setting up the UNESCO Open Science Portal, ensure your system meets these requirements:

2.2.1 System Requirements

- **Operating System:** macOS, Linux, or Windows with WSL2
- **Memory:** Minimum 4GB RAM (8GB recommended for optimal performance)
- **Storage:** At least 5GB free disk space
- **Internet:** Stable connection for downloading dependencies

2.2.2 Required Software

- **Python 3.9+:** Required for InvenioRDM

```
python3 --version # Should show Python 3.9 or higher.
```

- **Docker & Docker Compose:** For containerized services

```
docker --version && docker compose version
```

- **Git:** For version control

```
git --version
```

2.2.3 Port Availability

Ensure these ports are available (not used by other applications):

- **5000**: Development server
- **5432**: PostgreSQL database
- **9200**: OpenSearch
- **6379**: Redis cache
- **5672**: RabbitMQ message queue

2.2.4 Docker Configuration

- Docker Desktop must be running
- User must have permissions to run Docker commands
- At least 4GB of memory allocated to Docker

2.3 Quick Start

This project includes a Makefile for easy development workflow management. All commands automatically handle virtual environment activation.

2.3.1 Initial Setup

For first-time setup, initialize the project:

```
make init
```

This command will:

- Check and clean Docker environment
- Create a Python virtual environment (`.venv`)
- Install all required dependencies
- Setup containerized services (PostgreSQL, OpenSearch, Redis, RabbitMQ)
- Prepare the InvenioRDM instance

Note: If you encounter issues with services not starting (especially Redis), run `make check` first to clean up Docker, then retry `make init`.

2.3.2 Development Workflow

1. Start the development server:


```
make up
```

Visit <https://127.0.0.1:5000> in your browser.

2. (Optional) Reset with curated data:

After the server is running, you can optionally reset all records and load fresh data from various sources. First wait for some minutes after the first `make up` (to let Invenio finish the default data import), then run:

```
make tools-setup-env
```

and select `N` to generate a new token that will be used to interact with Invenio's API. After that, you can:

```
# Reset with Lens.org publications
make tools-reset FILE='openseience_tools/openseience_tools/sources/lens/data/publications.json'
```

This command:

- Deletes all existing records
- Imports fresh, curated data
- Gives you full programmatic control over your dataset
- Perfect for development and testing

3. Stop the server:

```
make stop
```

2.3.3 Login Credentials

The system comes with three ready-to-use user accounts:

Email	Password	Role	Description
<code>admin@unesco.org</code>	<code>Passw0rd!</code>	Administrator	Full system access including admin panel
<code>scientist@unesco.org</code>	<code>Passw0rd!</code>	Scientist	Standard user for creating and managing records
<code>demo@unesco.org</code>	<code>Passw0rd!</code>	Demo User	Basic user for demonstration purposes

Note: All users are pre-confirmed and active, so no email verification is required.

Stop all services:

```
make stop
```

Build assets (CSS, JS, templates):

```
make build
```

Complete cleanup (❗ destroys all data):

```
make destroy
```

2.4 Development

2.4.1 Making Changes

When customizing the portal, you can modify:

- **Templates:** `templates/` directory (HTML/Jinja2)
- **Styles:** `assets/less/` (LESS/CSS files)
- **JavaScript:** `assets/js/`
- **Configuration:** `invenio.cfg`
- **Static files:** `static/images/`

2.4.2 Build Assets

After modifying templates, styles, or JavaScript:

```
make build
```

Then refresh your browser (Ctrl+F5 or Cmd+Shift+R).

2.4.3 Development Loop

```
# 1. Start server (once)
make up

# 2. Make changes to templates/assets/styles

# 3. Build assets
make build

# 4. Refresh browser at https://127.0.0.1:5000

# Repeat steps 2-4 as needed
```

Note: Python code changes reload automatically, but assets require `make build`.

2.5 OpenScience Tools

The project includes a Python package for programmatic interaction with InvenioRDM.

2.5.1 Quick Setup

```
# 1. Start InvenioRDM
make up

# 2. Install the tools package
make tools-install

# 3. Generate API token and configure environment
make tools-setup-env
```

2.5.2 SDK Examples

Run Example Scripts:

```
# Single record demo (insert/update/delete)
make tools-examples

# Batch import all publications
make tools-examples EXAMPLE=batch_import
```

2.5.3 Common Operations

Search Records:

```
# Search all records
make tools-search

# Search with query
make tools-search QUERY='climate data'

# Search with options
make tools-search QUERY='test' OPTS='--detailed --size 20'
```

View Record Details:

```
# View specific record
make tools-view RECORD_ID='abc-123'

# View as JSON
make tools-view RECORD_ID='abc-123' OPTS='--format json'
```

Import Data:

```
# Import from Lens.org JSON export
make tools-import-lens
FILE='openscience_tools/openscience_tools/sources/lens/data/publications.json'

# Import with limit
make tools-import-lens
FILE='openscience_tools/openscience_tools/sources/lens/data/publications.json' OPTS='--limit 10'

# Dry run (preview without importing)
make tools-import-lens
FILE='openscience_tools/openscience_tools/sources/lens/data/publications.json' OPTS='--dry-run'
```

Delete Records:

```
# Preview deletions
make tools-cleanup OPTS='--dry-run'

# Delete all records (requires confirmation)
make tools-cleanup OPTS='--confirm'
```

Reset Data:

```
# Delete all records and import fresh data from Lens.org
make tools-reset FILE='openscience_tools/openscience_tools/sources/lens/data/publications.json'

# Reset with limited records
make tools-reset FILE='openscience_tools/openscience_tools/sources/lens/data/publications.json'
OPTS='--limit 10'
```

2.5.4 Available Commands

Command	Description
<code>make tools-install</code>	Install openscience_tools Python package
<code>make tools-setup-env</code>	Generate API token and configure environment
<code>make tools-search</code>	Search records (use QUERY='...')
<code>make tools-view</code>	View record details (use RECORD_ID='...')
<code>make tools-cleanup</code>	Delete all records
<code>make tools-import-lens</code>	Import from Lens.org (use FILE='...')
<code>make tools-reset</code>	Delete all + import from Lens

Command	Description
<code>make tools-help</code>	Show detailed help with examples

2.5.5 Override Credentials (Optional)

You can override the default credentials stored in `.env`:

```
# Use different InvenioRDM instance
make tools-search QUERY='test' BASE_URL='https://myinstance.org' TOKEN='my-token'

# Reset data on different instance
make tools-reset FILE='data.json' BASE_URL='https://myinstance.org' TOKEN='my-token'
```

2.5.6 Documentation

For complete documentation, see:

- `openscience_tools/README.md` - Full package documentation with installation, usage, and troubleshooting

2.6 Available Make Commands

Command	Description
<code>make init</code>	Initialize project with virtualenv and setup
<code>make users</code>	Create ready-to-use users with predefined passwords
<code>make up</code>	Start development server and services
<code>make stop</code>	Stop all services and processes
<code>make stop-all</code>	Force stop all services, containers and processes
<code>make build</code>	Build frontend assets
<code>make check</code>	Check and fix Docker services if needed
<code>make destroy</code>	Completely destroy instance and virtualenv
<code>make help</code>	Show available commands

2.7 Project Structure

Directory/File	Description
<code>Dockerfile</code>	Container image definition
<code>Pipfile</code> / <code>Pipfile.lock</code>	Python dependencies managed by pipenv
<code>app_data/</code>	Application data (vocabularies, pages)
<code>assets/</code>	Frontend assets (CSS, JavaScript, LESS, templates)
<code>docker/</code>	Docker configuration files
<code>invenio.cfg</code>	Main InvenioRDM configuration
<code>static/</code>	Static files served as-is
<code>templates/</code>	Custom Jinja2 templates
<code>.venv/</code>	Python virtual environment

2.8 Important Notes

- **SSL Certificate:** The development server uses a self-signed SSL certificate. Your browser will show a security warning that you need to bypass.
- **Virtual Environment:** All Invenio commands must be run within the activated virtual environment. The Makefile handles this automatically.
- **First Run:** Initial setup and first server start may take several minutes as Docker images are downloaded.

2.9 Customizations

This UNESCO Science Portal instance includes:

- Custom UNESCO branding and colors
- Inter font family (UNESCO's official typeface)
- Custom footer with UNESCO links
- Favicon and logo customizations

2.10 Documentation

For detailed InvenioRDM documentation, visit:

- [InvenioRDM Documentation](#)
- [InvenioRDM CLI Reference](#)

3 ▯ UNESCO Open Science Portal - Kubernetes Deployment

Complete guide for deploying InvenioRDM on **Kind (Kubernetes in Docker)** for local development and testing.

3.1 ▯ Table of Contents

1. [Overview](#)
 2. [Prerequisites](#)
 3. [Step-by-Step Guide](#)
 4. [Data Import](#)
 5. [Daily Operations](#)
 6. [Architecture](#)
-

3.2 ▯ Overview

This deployment uses:

- **Kind:** Local Kubernetes cluster running in Docker
- **Official Helm Chart:** InvenioRDM from `inveniosoftware/helm-invenio` (v0.8.1)
- **External Services:** PostgreSQL, Redis, RabbitMQ, OpenSearch with public images
- **Custom Image:** Your customized InvenioRDM with compiled assets
- **HTTP Mode:** Simplified for local development (no TLS)
- **Pinned Versions:** Helm chart versions are fixed to prevent breaking changes

3.2.1 Why This Approach?

- ▯ **Production-ready:** Uses official Helm chart
 - ▯ **No registry issues:** Public images instead of Bitnami registry
 - ▯ **Fast iteration:** Rebuild and redeploy quickly
 - ▯ **Complete control:** Full access to all components
 - ▯ **Easy cleanup:** Delete and recreate in minutes
-

3.3 ▯ Prerequisites

3.3.1 Required Tools

Install these tools before starting:

```
# macOS
brew install kind kubectl helm docker

# Verify installation
kind version
kubectl version --client
helm version
docker --version
```

3.3.2 System Requirements

- **RAM:** 8GB minimum (16GB recommended)
- **Disk:** 10GB free space
- **Docker:** Running with at least 6GB memory allocated

3.3.3 Check Everything

From the `k8s` directory:

```
cd k8s
make kind-check
```

This verifies all tools are installed and shows their versions.

3.4 ☑ Step-by-Step Guide

Following the steps to deploy the app.

3.4.1 Step 1: Create Kind Cluster

```
make kind-create
```

What it does:

- Creates Kind cluster named `unesco-rdm`
- Sets up port mappings (80 → 8080, 443 → 8443)
- Installs Ingress NGINX controller
- Creates persistent volume directories

Time: ~2 minutes

Verify:

```
kind get clusters
kubectl cluster-info --context kind-unesco-rdm
```

3.4.2 Step 2: Deploy External Services

```
make kind-deploy-services
```

What it does:

- Creates `unesco-rdm` namespace
- Deploys PostgreSQL (database)
- Deploys Redis (cache)
- Deploys RabbitMQ (message queue)
- Deploys OpenSearch (search engine)
- Waits for all services to be ready

Time: ~2-3 minutes

Verify:

```
kubectl get pods -n unesco-rdm
# All pods should show "Running"
```

3.4.3 Step 3: Build and Load Docker Image

```
make kind-load-image
```

What it does:

- Builds custom InvenioRDM image from `Dockerfile`
- Compiles webpack assets during build
- Loads image into Kind cluster
- Tags as `sc-openscience:latest`

Time: ~5-10 minutes (first build)

Verify:

```
docker exec -it unesco-rdm-control-plane crictl images | grep sc-openscience
```

3.4.4 Step 4: Deploy InvenioRDM with Helm

```
make kind-deploy-helm
```

What it does:

- Adds InvenioRDM Helm repository
- Installs chart with custom values (`values.yaml`)
- Deploys web, workers, and beat pods
- Configures to use external services
- Sets HTTP-only mode for local dev

Time: ~2-3 minutes

Verify:

```
kubectl get pods -n unesco-rdm
# Should see: web, worker (2 replicas), worker-beat
```

3.4.5 Step 5: Initialize System

```
make kind-init
```

What it does:

- Loads InvenioRDM vocabularies (resource types, languages, licenses, etc.)
- Creates database tables
- Sets up search indices
- Creates file storage location
- **Important:** Vocabularies are loaded by workers in background (~2-3 minutes)

Time: ~3-5 minutes

Verify:

```
# Check vocabularies loaded
kubectl exec -n unesco-rdm deployment/unesco-rdm-invenio-web -c web -- \
  invenio shell -c "from invenio_vocabularies.proxies import current_service;
print(current_service.read_all(identity=system_identity, type='resourcetypes').total)"
# Should show: 45
```

3.4.6 Step 6: Create Admin User

```
make kind-create-admin
```

What it does:

- Creates user `admin@unesco.org` with password `admin123`
- Assigns admin role and superuser permissions
- Generates API token
- Saves token to `../openscience_tools/config/.env.kind`

Time: ~30 seconds

Credentials:

- Email: `admin@unesco.org`
 - Password: `admin123`
-

3.4.7 Step 7: Access Application

```
# Start port forwarding (keep this running)
make kind-port-forward
```

Access URLs:

- Web UI: `http://localhost:8080`
- API: `http://localhost:8080/api`

Login with admin credentials above.

3.5 ☒ Data Import

3.5.1 Quick Reset with Lens Data

Delete all records and import 26 publications from Lens.org:

```
make kind-reset-lens
```

Time: ~2-3 minutes

3.6 □ Daily Operations

3.6.1 Check Status

```
# Overall status
make kind-status

# Just pods
kubectl get pods -n unesco-rdm

# Watch pods (live updates)
kubectl get pods -n unesco-rdm -w
```

3.6.2 View Logs

```
# Web logs (follow mode)
make kind-logs

# Worker logs
kubectl logs -f deployment/unesco-rdm-invenio-worker -n unesco-rdm

# Beat logs
kubectl logs -f deployment/unesco-rdm-invenio-worker-beat -n unesco-rdm

# Specific pod
kubectl logs <pod-name> -n unesco-rdm --all-containers
```

3.6.3 Restart Application

```
# Restart all deployments
make kind-restart

# Restart specific component
kubectl rollout restart deployment/unesco-rdm-invenio-web -n unesco-rdm
kubectl rollout restart deployment/unesco-rdm-invenio-worker -n unesco-rdm
```

3.6.4 Execute Commands

```
# Open shell in web pod
make kind-shell

# Run single command
kubectl exec -n unesco-rdm deployment/unesco-rdm-invenio-web -c web -- invenio --help

# Create user
kubectl exec -n unesco-rdm deployment/unesco-rdm-invenio-web -c web -- \
  invenio users create test@example.com --password test123 --active

# List records
kubectl exec -n unesco-rdm deployment/unesco-rdm-invenio-web -c web -- \
  invenio shell -c "from invenio_rdm_records.proxies import current_rdm_records_service;
print(current_rdm_records_service.search(system_identity).total)"
```

3.6.5 Rebuild After Code Changes

```
# 1. Make your code changes in parent directory
cd ..
# edit files...

# 2. Rebuild and reload image
cd k8s
make kind-load-image

# 3. Restart pods to use new image
make kind-restart

# 4. Check if running
make kind-status
```

3.6.6 Remove Deployment (Keep Cluster)

```
make kind-clean
```

This removes the Helm release and namespace but keeps the Kind cluster. Useful for quick redeployment.

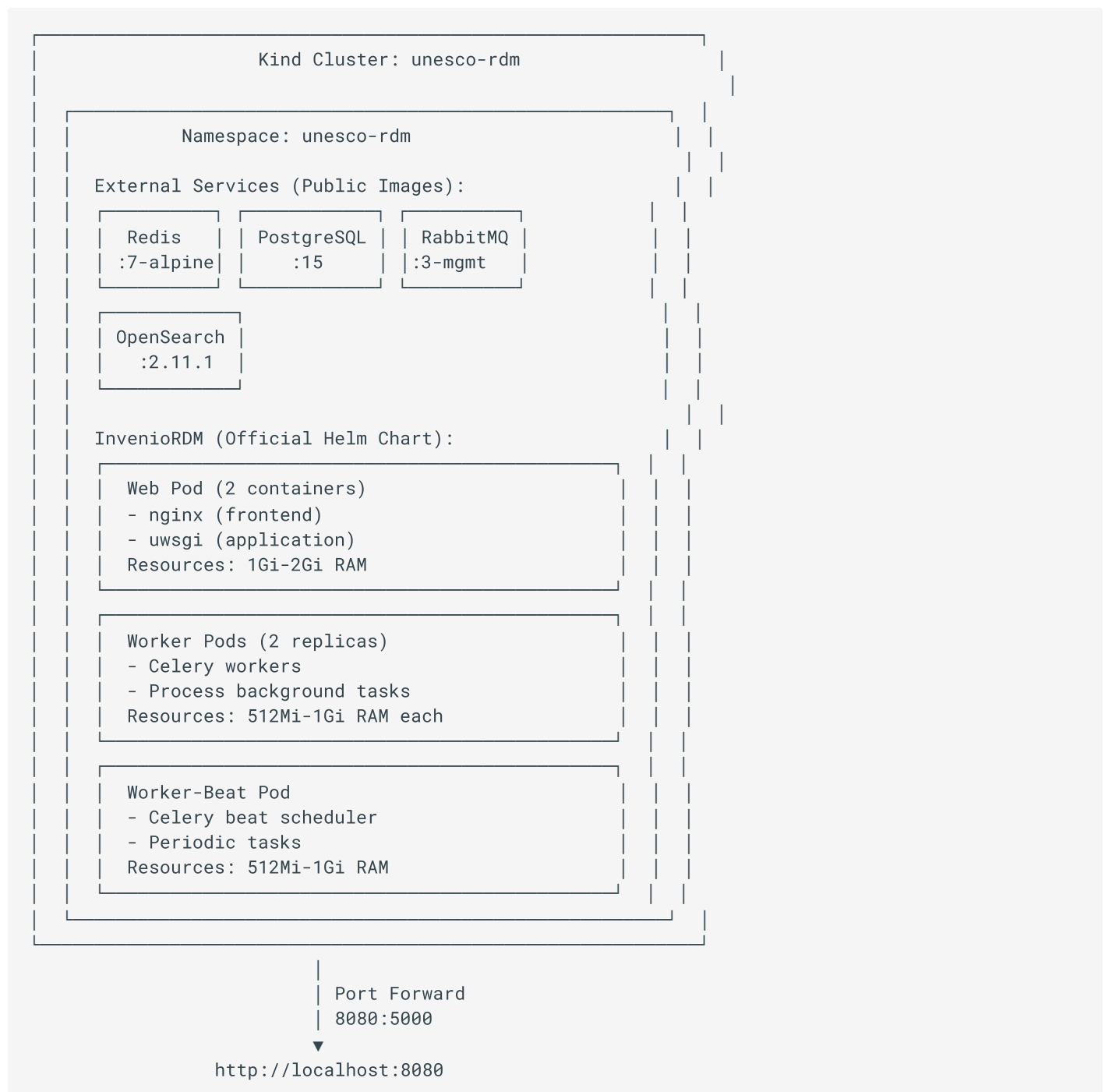
3.6.7 Complete Cleanup

```
make kind-down
```

This deletes the entire Kind cluster. All data is lost.

3.7 ☒ Architecture

3.7.1 Component Diagram



3.7.2 Resource Allocation

Component	CPU Request	CPU Limit	Memory Request	Memory Limit
Web	500m	1000m	1Gi	2Gi
Worker (x2)	250m	500m	512Mi	1Gi

Component	CPU Request	CPU Limit	Memory Request	Memory Limit
Worker-Beat	200m	500m	512Mi	1Gi
PostgreSQL	250m	500m	512Mi	1Gi
Redis	100m	200m	256Mi	512Mi
RabbitMQ	200m	500m	512Mi	1Gi
OpenSearch	500m	1000m	1Gi	2Gi
Total	~2.5 CPU	~5 CPU	~4.5Gi	~9.5Gi

Recommendation: Allocate at least 8GB RAM to Docker Desktop.

3.8 Configuration Files

3.8.1 `kind-config.yaml`

- Kind cluster configuration
- Port mappings (80 → 8080, 443 → 8443)
- Node settings

3.8.2 `external-services.yaml`

- Deployments for PostgreSQL, Redis, RabbitMQ, OpenSearch
- Uses public Docker images
- Single replica with emptyDir volumes (not persistent)

3.8.3 `values.yaml`

- Helm chart overrides
- HTTP-only configuration
- External service connections
- Resource limits
- Probe configurations (disabled/adjusted)

3.8.4 Makefile

- All deployment commands
 - Encapsulates kubectl and helm operations
 - Provides convenient targets
-

3.9 ☒ Make Targets Reference

3.9.1 Setup & Deployment

Command	Description
<code>make kind-check</code>	Verify all tools installed
<code>make kind-create</code>	Create Kind cluster
<code>make kind-deploy-services</code>	Deploy external services
<code>make kind-load-image</code>	Build and load Docker image
<code>make kind-deploy-helm</code>	Install Helm chart
<code>make kind-init</code>	Initialize database & vocabularies
<code>make kind-create-admin</code>	Create admin user
<code>make kind-full-deploy</code>	Complete deployment (all above)

3.9.2 Operations

Command	Description
<code>make kind-status</code>	Show cluster status
<code>make kind-logs</code>	Follow web pod logs
<code>make kind-shell</code>	Open shell in web pod

Command	Description
<code>make kind-port-forward</code>	Forward port 8080 → 5000
<code>make kind-restart</code>	Restart all deployments

3.9.3 Data Management

Command	Description
<code>make kind-reset-lens</code>	Delete all + import Lens data

3.9.4 Version Management

Command	Description
<code>make kind-helm-versions</code>	Check available Helm chart versions

3.9.5 Cleanup

Command	Description
<code>make kind-clean</code>	Remove deployment (keep cluster)
<code>make kind-delete</code>	Delete Kind cluster
<code>make kind-down</code>	Complete cleanup (clean + delete)

3.10 ☒ Additional Resources

- [InvenioRDM Documentation](#)
 - [Helm Chart Repository](#)
 - [Kind Documentation](#)
 - [Kubernetes Basics](#)
-

3.11 ☒ Getting Help

1. **Check logs:** `make kind-logs`
2. **Check status:** `make kind-status`
3. **Check InvenioRDM docs:** <https://inveniordm.docs.cern.ch/>

Happy deploying! ☒

4 UNESCO Open Science Portal - Production Deployment Guide

This guide covers deploying the UNESCO Open Science Portal (InvenioRDM) to production or staging environments.

4.1 Table of Contents

1. [Overview](#)
 2. [Pre-Deployment Checklist](#)
 3. [Deployment Steps](#)
 4. [Verification](#)
 5. [Troubleshooting](#)
 6. [Rollback Procedure](#)
 7. [Quick Reference](#)
-

4.2 Overview

The UNESCO Open Science Portal includes:

CMS System: - Database-driven Content Management System - Static pages management (About, Privacy, Natural Sciences Family, etc.) - Admin interface at `/administration/cms` - Multilingual support with fixtures

Custom Fields & Facets: - `publication:year` - Publication year for faceted search - `publication:country` - Countries from author affiliations - `publication:funding_org` - Funding organizations - `publication:is_open_access` - Boolean flag for open access status - `publication:open_access_colour` - Color category (gold, green, hybrid, bronze)

Technical Stack: - InvenioRDM with custom site package - Docker Compose for container orchestration - PostgreSQL database with Alembic migrations - Swiper.js for carousels (loaded from CDN)

4.3 Pre-Deployment Checklist

Before starting deployment, ensure you have:

- ✓ SSH access to the deployment server
- ✓ Docker and Docker Compose installed and running

- ✓ Database backup completed (see below)
- ✓ Sufficient disk space (minimum 5GB free)
- ✓ Access to environment variables and secrets

Create Database Backup:

```
# Create timestamped database backup
docker compose exec db pg_dump -U invenio invenio | gzip > backup_$(date +%Y%m%d_%H%M%S).sql.gz

# Verify backup file exists and has size > 0
ls -lh backup_*.sql.gz
```

4.4 Deployment Steps

4.4.1 Step 1: Pull Latest Changes

SSH into the server and pull the latest code:

```
# SSH into the server
ssh <server>

# Navigate to the project directory
cd /opt/unesco-science-portal # or your DEPLOY_PATH

# Pull the latest changes
git fetch origin
git checkout dev # or main/production depending on your branch
git pull origin dev
```

4.4.2 Step 2: Update Configuration

⚠ **CRITICAL NOTES:** - The `invenio.cfg` file contains environment-specific secrets and is **NOT** tracked in Git - The file is typically a **bind mount** from the host filesystem (e.g., `/opt/dockerapps/sc-openscience-dev/invenio.cfg` → `/opt/invenio/var/instance/invenio.cfg`) - Always backup the **host file** before making changes - After pulling new code, you must manually update this file on the server

2.1 Backup Current Configuration

Always backup before making changes:

```
# Create timestamped backup inside the container
docker compose exec web-ui cp /opt/invenio/var/instance/invenio.cfg \
/opt/invenio/var/instance/invenio.cfg.backup_$(date +%Y%m%d_%H%M%S)

# Verify backup was created
docker compose exec web-ui ls -la /opt/invenio/var/instance/invenio.cfg*

# Optional: Copy backup to local machine for extra safety
docker cp $(docker compose ps -q web-ui):/opt/invenio/var/instance/invenio.cfg \
./invenio.cfg.backup_$(date +%Y%m%d_%H%M%S)
```

▮ **Keep note of the backup filename** - you'll need it if you need to rollback!

2.2 Update Configuration

Choose the appropriate method:

Option A: Generate from Template (Fresh Deployments)

If you have environment variables set up:

```
# Generate invenio.cfg from template with environment variable substitution
envsubst < invenio.cfg.template > invenio.cfg.NEW

# Review the generated file to ensure secrets were substituted correctly
grep -E "DATABASE|SECRET|PASSWORD" invenio.cfg.NEW | head -5

# Copy to container if secrets look correct
docker cp invenio.cfg.NEW $(docker compose ps -q web-ui):/opt/invenio/var/instance/invenio.cfg

# Clean up
rm invenio.cfg.NEW
```

Option B: Manually Patch Existing Config (Existing Deployments)

If you have a working configuration with secrets already in place:

Step B.1: Enter the container and edit the config:

```
# Enter the container
docker compose exec web-ui bash

# Edit the config file (use vim, nano, or preferred editor)
vi /opt/invenio/var/instance/invenio.cfg
```

Step B.2: Edit what you want to change

Step B.3: Save and exit (in vim: `ESC` → `:wq` → `Enter`)

Step B.4: Exit container and verify:

```
exit

# Verify configuration was updated
docker compose exec web-ui grep "cdn.jsdelivr.net" /opt/invenio/var/instance/invenio.cfg
```

4.4.3 Step 3: Rebuild Docker Images

```
# Rebuild the application images
docker compose build web-ui web-api

# OR if using pre-built CI images:
docker compose pull
```

4.4.4 Step 4: Update Database Schema

This step creates the CMS database tables if they don't exist.

```
# Step 4.1: Reinstall the site package
docker compose exec web-ui pip install -e /opt/invenio/src/site/

# Step 4.2: Check current migration status
docker compose exec web-ui invenio alembic current

# Step 4.3: Stamp base dependency (if needed)
docker compose exec web-ui invenio alembic stamp dbdbc1b19cf2 || true

# Step 4.4: Check if CMS tables already exist
docker compose exec web-ui invenio shell -c "
from invenio_db import db
result = db.session.execute(db.text(\"SELECT COUNT(*) FROM pg_tables WHERE
tablename='cms_content'\"))
print('CMS table exists:', result.scalar() == 1)
"

# Step 4.5: Apply CMS migrations
# If CMS tables DO NOT exist (new deployment):
docker compose exec web-ui invenio alembic upgrade my_site@head

# If CMS tables ALREADY exist (update deployment):
docker compose exec web-ui invenio alembic stamp my_site@head
```

Verify CMS tables were created:

```
docker compose exec web-ui invenio shell -c "
from invenio_db import db
result = db.session.execute(db.text(\"SELECT table_name FROM information_schema.tables WHERE
table_name LIKE 'cms_%'\"))
for row in result:
    print(row[0])
"
```

Expected output:

```
cms_content
```

4.4.5 Step 5: Initialize Custom Fields

⚠ **CRITICAL STEP:** This registers new custom fields in the database.

```
# Initialize custom fields (required for new publication:* fields)
docker compose exec web-ui invenio rdm-records custom-fields init
```

Expected output:

```
Initializing custom fields...
lens:id
lens:open_access
lens:external_ids
lens:source
lens:references
lens:mesh_terms
lens:scholarly_citations
lens:chemicals
publication:open_access_colour
publication:is_open_access
publication:year
publication:country
publication:funding_org
Custom fields initialized successfully.
```

4.4.6 Step 6: Rebuild Search Indices

⚠ **CRITICAL STEP:** New facets require updated index mappings.


```
# 1. Destroy existing indices (removes old mappings)
docker compose exec web-ui invenio index destroy --force --yes-i-know

# 2. Create indices with new mappings
docker compose exec web-ui invenio index init

# 3. Rebuild all record indices (re-index all records)
docker compose exec web-ui invenio rdm-records rebuild-index
```

⚠ **Note:** Step 3 may take several minutes depending on record count. Search will be unavailable during this process.

Verify indices:

```
# List indices
docker compose exec web-ui invenio index list

# Check if records were indexed
docker compose exec web-ui invenio shell -c "from invenio_search import current_search_client;
print(current_search_client.count(index='*records*))"
```

4.4.7 Step 7: Load CMS Content

Load default content for footer, header, and static pages:

```
# Load CMS fixtures (overwrites existing content)
docker compose exec web-ui invenio cms load-fixtures --force
```

Verify fixtures were loaded:

```
docker compose exec web-ui invenio shell -c "
from invenio_db import db
result = db.session.execute(db.text(\"SELECT resource_type, lang, COUNT(*) FROM cms_content
GROUP BY resource_type, lang\"))
for row in result:
    print(f'{row[0]} ({row[1]}): {row[2]} entries')
"
```

Expected output:

```
header_frontpage (en): 1 entries
footer (en): 1 entries
static_page (en): 3 entries
```

4.4.8 Step 8: Build Frontend Assets

```
# Build CSS and JavaScript assets (use invenio, not invenio-cli in containers)
docker compose exec web-ui invenio webpack buildall
docker compose exec web-ui invenio-cli assets build
```

If assets fail to build:

```
# Clean and rebuild
docker compose exec web-ui invenio webpack clean
docker compose exec web-ui invenio webpack buildall

# If you encounter memory issues (exit code -9), increase Node.js memory:
docker compose exec \
  -e NODE_OPTIONS="--max-old-space-size=3072" \
  web-ui invenio webpack buildall
```

4.4.9 Step 9: Collect Static Files

```
docker compose exec web-ui invenio collect -v
```

4.4.10 Step 10: Restart Services

```
# Restart all InvenioRDM services
docker compose restart web-ui web-api worker

# OR perform full restart
docker compose down
docker compose up -d
```

4.5 Verification

4.5.1 Check Custom Fields Configuration

```
# Verify custom fields are registered
docker compose exec web-ui invenio shell -c "from flask import current_app; print([f.name for f
in current_app.config.get('RDM_CUSTOM_FIELDS', [])])"
```

Expected: Should list all custom fields including `publication:year`, `publication:country`,
`publication:funding_org`, `publication:is_open_access`, `publication:open_access_colour`

4.5.2 Check Facets in API

```
# Test facets/aggregations
curl -s "https://your-domain.org/api/records?size=1" | jq '.aggregations'
```

Expected: Should include aggregations for: - publication_year - publication_country - funding_org - is_open_access

4.5.3 Check Services Health

```
# Check all containers are running
docker compose ps

# Check logs for errors
docker compose logs --tail=50 web-ui
docker compose logs --tail=50 web-api
docker compose logs --tail=50 worker
```

4.5.4 Verify Frontend Changes

Open in browser and verify:

1. **Homepage:** https://your-domain.org/
2. UNESCO-styled page with carousels
3. Highlight cards carousel works (Swiper.js)
4. Research cards carousel works
5. **Search Page:** https://your-domain.org/search
6. Custom facets displayed in left sidebar
7. Open Access toggle works
8. Facets: Resource Type, Year, Country, Funding Organization
9. **Record Detail:** Click on any record
10. Collapsible metadata panels display
11. UNESCO styling applied

4.5.5 Verify Static Pages

```
# Test About page
curl -s "https://your-domain.org/pages/about" | grep -i "unesco" | head -5

# Test Privacy page
curl -s "https://your-domain.org/pages/privacy" | grep -i "privacy" | head -5
```

4.6 Troubleshooting Guide

4.6.1 Issue: Swiper.js Carousel Not Working

Symptoms: Carousels not sliding, console shows CSP errors

Solution: Verify CSP headers include `https://cdn.jsdelivr.net`

```
docker compose exec web-ui grep -A 20 "content_security_policy"
/opt/invenio/var/instance/invenio.cfg
```

4.6.2 Issue: CMS Migration Fails

Symptoms: `alembic upgrade` fails with "relation already exists" or "branch not found"

Solution:

```
# Check current state
docker compose exec web-ui invenio alembic current

# If my_site branch doesn't exist, create it
docker compose exec web-ui invenio alembic stamp --purge my_site@head

# If tables already exist but migration thinks they don't
docker compose exec web-ui invenio alembic stamp my_site@head
```

4.6.3 Issue: Static Pages Return 404

Symptoms: `/pages/about` returns 404

Solution: Check if fixtures were loaded

```
# Check loaded pages
docker compose exec web-ui invenio shell -c "
from my_site.models.cms import CMSContent
from invenio_db import db
pages = db.session.query(CMSContent).filter_by(resource_type='static_page').all()
for p in pages:
    print(f'{p.slug}: {p.title}')
"

# If empty, reload fixtures
docker compose exec web-ui invenio cms load-fixtures --force
```

4.6.4 Issue: Assets Not Loading (404 on CSS/JS)

Symptoms: Page loads but looks unstyled

Solution: Rebuild and collect assets

```
# Rebuild and collect assets
docker compose exec web-ui invenio webpack clean
docker compose exec web-ui invenio webpack buildall
docker compose exec web-ui invenio collect -v

# Restart nginx if used
docker compose restart nginx
```

4.6.5 Issue: Database Connection Errors

Symptoms: Application fails to start, database connection refused

Solution:

```
# Check PostgreSQL is running
docker compose ps db

# Check database logs
docker compose logs db

# Restart database
docker compose restart db

# Wait for database to be ready
docker compose exec web-ui invenio db init
```

4.7 Rollback Procedure

If something goes wrong during deployment:

4.7.1 Rollback Configuration

```
# Restore previous configuration (use your backup timestamp)
docker compose exec web-ui cp \
  /opt/invenio/var/instance/invenio.cfg.backup_YYYYMMDD_HHMMSS \
  /opt/invenio/var/instance/invenio.cfg
```

4.7.2 Rollback Code

```
# Revert to previous git commit
git log --oneline -10 # Find the previous commit hash
git checkout <previous-commit-hash>

# Rebuild images
docker compose build web-ui web-api

# Restart services
docker compose restart web-ui web-api worker
```

4.7.3 Rollback Database Migration

```
# Rollback last migration
docker compose exec web-ui invenio alembic downgrade my_site@-1

# Rollback to specific revision
docker compose exec web-ui invenio alembic downgrade <revision-id>
```

4.7.4 Restore Database from Backup

```
# Stop services
docker compose down

# Restore database
gunzip < backup_YYYYMMDD_HHMMSS.sql.gz | docker compose exec -T db psql -U invenio invenio

# Start services
docker compose up -d
```

4.8 Quick Reference

4.8.1 Complete Deployment Sequence

Copy-paste friendly command sequence for standard deployment:

```

# Navigate to project
cd /opt/unesco-science-portal

# Pull latest code
git pull origin dev

# Backup configuration
docker compose exec web-ui cp /opt/invenio/var/instance/invenio.cfg \
    /opt/invenio/var/instance/invenio.cfg.backup_$(date +%Y%m%d_%H%M%S)

# Update configuration (if needed)
# ... manually edit or use envsubst ...

# Rebuild images
docker compose build web-ui web-api

# Update site package and database
docker compose exec web-ui pip install -e /opt/invenio/src/site/
docker compose exec web-ui invenio alembic upgrade my_site@head

# Initialize custom fields (CRITICAL!)
docker compose exec web-ui invenio rdm-records custom-fields init

# Rebuild search indices (CRITICAL!)
docker compose exec web-ui invenio index destroy --force --yes-i-know
docker compose exec web-ui invenio index init
docker compose exec web-ui invenio rdm-records rebuild-index

# Load CMS content
docker compose exec web-ui invenio cms load-fixtures --force

# Build assets
docker compose exec web-ui invenio webpack buildall
docker compose exec web-ui invenio collect -v

# Restart services
docker compose restart web-ui web-api worker

# Verify deployment
docker compose ps
docker compose logs --tail=20 web-ui

```

4.8.2 Useful Commands

```
# Check service status
docker compose ps

# View logs
docker compose logs -f web-ui
docker compose logs -f worker

# Enter web container
docker compose exec web-ui bash

# Run Invenio commands
docker compose exec web-ui invenio <command>

# Database shell
docker compose exec web-ui invenio shell

# Check migration status
docker compose exec web-ui invenio alembic current
docker compose exec web-ui invenio alembic history

# Rebuild assets
docker compose exec web-ui invenio webpack clean
docker compose exec web-ui invenio webpack buildall
docker compose exec web-ui invenio collect -v
```
